Package 'levygof'

November 13, 2026

Title Goodness-of-Fit Tests for Levy Degradation Processes

Version 0.1.0

Description Goodness-of-fit tests for parametric Levy degradation processes (gamma, Wiener, inverse Gaussian, variance-gamma and Meixner processes) from one or several trajectories. With regular inspections the observed increments are tested; with irregular inspections the parameters are estimated by maximum likelihood (package 'levyfit'), the trajectories are resampled on a regular grid by stochastic bridges (package 'levyBridge', Grall-Maes (2012) <doi:10.1177/1748006X12462522>) and the reconstructed increments are tested with tests for independent and identically distributed samples.

License GPL-3

URL https://github.com/rytracetito/levy_packages/tree/main/levygof, https://github.com/rytracetito/levy_packages

BugReports https://github.com/rytracetito/levy_packages/issues

Encoding UTF-8

Language en

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.3

Imports DegradSim, fBasics, gofgamma, gofIG, goft, gsl, levyBridge, levyfit, nortest, Runuran, stats, tseries

Suggests testthat (>= 3.0.0)

Remotes github::rytracetito/levy_packages/levyfit, github::rytracetito/levy_packages/levyBridge, github::rytracetito/levy_packages/DegradSim

Config/testthat/edition 3

Author Amedée Rytrace Tito [aut, cre], Christian Paroissin [aut]

Maintainer Amedée Rytrace Tito <rytracetito5@gmail.com>

Contents

levygof-package 2
gof.levy 4
tests_disponibles 9
test_adequation 13

Index 17

`levygof-package` *levygof: goodness-of-fit tests for Levy degradation processes*

Description

`levygof` tests whether degradation trajectories come from a parametric Levy process with unknown parameters:

- gamma (`"HGP"`);
- Wiener (`"HWP"`);
- inverse Gaussian (`"InvGaus"`);
- variance-gamma (`"VG"`);
- Meixner (`"Meixner"`).

Details

The main function is `gof.levy()`.

Functions of the package

Click on a function name for its help page:

Function	Input	Role
`gof.levy()`	one or several trajectories	estimation, bridges if needed, tests
`test_adequation()`	i.i.d. sample of increments	tests only (steps 5 and 6 below)
`tests_disponibles()`	name of a model, or nothing	list of the tests of each model

Inspection times following a chosen pattern, and simulated trajectories to try the tests, are given by the package `DegradSim` (`DegradSim::inspection_times()` and its simulation functions).

Methodology

A Levy process has independent and stationary increments: the increments over a same time step are i.i.d. The test proceeds in six steps.

1. **Estimate** the parameters from the observed trajectories by maximum likelihood (package `levyfit`).
2. **Distribution of the bridge** of the process between two observations (package `levyBridge`).
3. **Simulate the bridge** with the estimated parameters.
4. **Pseudo-sample on a regular grid**: each trajectory is resampled on a regular grid, with the method `"Bridge"` (Grall-Maës, 2012, one grid per trajectory), `"ExtBridge"` (grid containing all the inspection times, step common to all the trajectories) or `"MultiBridge"` (one grid of Grall-Maës type common to all the trajectories). With several trajectories, `"ExtBridge"` and `"MultiBridge"` give one i.i.d. sample for all the trajectories together. The grid, the step and the drawing of the bridge of each method are detailed in the section Resampling methods of `gof.levy()`.

5. **Choose a test** for an i.i.d. sample of the distribution of the increments (list of the tests: `tests_disponibles()`).
6. **Apply the test** to the reconstructed increments.

With regular inspections the observed increments are already i.i.d.: steps 1 to 4 are skipped. The function `gof.levy()` does all the steps, and the function `test_adequation()` does steps 5 and 6 for a sample of increments.

Data

`gof.levy()` accepts the formats of `levyfit` and `levyBridge`: a data frame (times, levels) for one trajectory, a list of such data frames for several trajectories with their own times, or a matrix (times, levels of each trajectory), as returned by the simulation functions of `DegradSim`.

Results

- `res` prints the model, the type of inspections, the time step, the estimated parameters and one row per test;
- `res$results` is the data frame of the tests (statistic, p-value, critical value, decision);
- `res$bridge` contains the resampled trajectories and `res$increments` the tested increments.

Author(s)

Maintainer: Amedée Rytrace Tito <rytracetito5@gmail.com>

Authors:

- Christian Paroissin <christian.paroissin@univ-pau.fr>

References

Grall-Maës, E. (2012). Use of the Kolmogorov-Smirnov test for gamma process. *Proceedings of the Institution of Mechanical Engineers, Part O: Journal of Risk and Reliability*, 226(6), 624-634. doi:10.1177/1748006X12462522

See Also

Useful links:

- https://github.com/rytracetito/levy_packages/tree/main/levygof
- https://github.com/rytracetito/levy_packages
- Report bugs at https://github.com/rytracetito/levy_packages/issues

Examples

```
# 1. A gamma trajectory (a = 2, b = 1) with 400 irregular inspections,
#    more and more frequent
set.seed(1)
t <- DegradSim::inspection_times("decreasing", n = 400, tmax = 400, digits = 1)$time
x <- DegradSim::sim_gamma(t, a = 2, b = 1)

# 2. Is it a gamma process? Is it a Wiener process?
gof.levy(x, "HGP", test = c("KS", "AD"), B = 100)
gof.levy(x, "HWP")
```

```
# 3. The tests of each model
tests_disponibles()
```

`gof.levy` *Goodness-of-fit test of a Levy degradation process*

Description

Tests whether one or several degradation trajectories come from a given Levy process (gamma, Wiener, inverse Gaussian, variance-gamma or Meixner), with unknown parameters. With regular inspections the observed increments are tested; with irregular inspections the parameters are estimated, the trajectories are resampled on a regular grid by stochastic bridges, and the reconstructed increments are tested.

Usage

```
gof.levy(
  data,
  model,
  test = NULL,
  method = "Bridge",
  option = 1,
  alpha = 0.05,
  B = 200,
  digits = NULL
)
```

Arguments

`data`	The observed trajectories, in one of the formats of the packages `levyfit` and `levyBridge`: • one trajectory: a data frame whose first column contains the inspection times and second column the degradation levels; • several trajectories, each with its own inspection times: a list of such data frames; • several trajectories with the same inspection times: a matrix whose first column contains the times and the other columns the levels, for instance the result of a simulation function of `DegradSim`. The columns are read by position, not by name. The inspection times of a trajectory must be strictly increasing; they do not have to start at 0 (the first inspection is taken as origin).
`model`	`"HGP"` (gamma process), `"HWP"` (Wiener process), `"InvGaus"` (inverse Gaussian process), `"VG"` (variance-gamma process) or `"Meixner"` (Meixner process). Upper or lower case does not matter.
`test`	Names of the tests to apply; `NULL` (default) applies all the tests of the model except the slow test BG of the inverse Gaussian model, see `tests_disponibles()`.
`method`	Resampling method of the bridges, used with irregular inspections only (see levyBridge::levyBridge-package):

- "Bridge" (default, Grall-Maës, 2012): one regular grid per trajectory, with as many points as observations;
- "ExtBridge": regular grid containing all the inspection times, with a time step common to all the trajectories;
- "MultiBridge": one regular grid common to all the trajectories, of step (total observed duration) / (total number of increments), with about as many points as observations.

option — Used only with method = "MultiBridge": 1 (default) to resample each trajectory up to its own last inspection, 2 to resample all the trajectories up to the last inspection time over all of them, the shorter ones being extended by increments of the process drawn with the estimated parameters (these values do not come from the data and lower the power; option 1 is recommended). See the section Resampling methods.

alpha — Level of the tests, strictly between 0 and 1.

B — Number of bootstrap replications of the tests that use a parametric bootstrap.

digits — Used only with method = "ExtBridge": number of decimal places of the inspection times (measured from the first inspection), which sets the step of the grid. NULL (default) takes the number of decimal places of the times, between 0 and 6; for times with more decimal places, for instance drawn at random, give digits: the times are then truncated to digits decimal places, with a warning. See the section Resampling methods of levyBridge::levyBridge-package.

Value

An object of class "gof.levy", printed by print(): a list with

- model, method (NA when no bridge was used), alpha, B;
- trajectories: number of trajectories;
- regular: TRUE when all the trajectories are inspected with a constant time step;
- out_of_support: TRUE when an observed increment is negative or zero under an increasing model (gamma, inverse Gaussian): H0 is then rejected for sure by all the tests, without computation, and print() shows this conclusion in one line (results keeps one row per test);
- parameters: maximum likelihood estimates of the parameters of the process (NULL when no bridge was used);
- steps: time step of the increments of each trajectory;
- bridge: the resampled trajectories (list of data frames with columns t and X, on the time scale of the data), or NULL;
- increments: data frame of the tested increments, with columns id (trajectory), step and dx;
- results: data frame of the tests, with a column group ("all" for the test on all the trajectories together, or the number of the trajectory) followed by the columns of test_adequation().

Method

The null hypothesis H0 is: the trajectories are independent trajectories of the Levy process model, with unknown parameters.

1. **Regular inspections** (each trajectory inspected with a constant time step Δ): the increments are i.i.d. with the distribution of X_Δ; they are tested directly.

2. **Irregular inspections**: the parameters are estimated by maximum likelihood on all the trajectories (package `levyfit`), then each trajectory is resampled on a regular grid by exact simulation of the bridge of the process with these parameters (package `levyBridge`, method `method`). Under H0 the reconstructed increments are i.i.d. with the distribution of $X_{t_{eq}}$, where t_{eq} is the time step of the grid; they are tested.
3. **Gamma and inverse Gaussian models**: these processes are increasing. A negative or zero observed increment is impossible under H0, which is then rejected for sure (p-value 0), without estimation nor bridge.

The tests are tests for i.i.d. samples with estimated parameters, see `test_adequation()` and `tests_disponibles()`.

Resampling methods

With irregular inspections, each trajectory is resampled on a regular grid by the package `levyBridge`, with the parameters estimated by `levyfit`. The three methods differ only by the grid. In all of them, the grid is covered from left to right: a grid point equal to an inspection time keeps the observed value, and any other grid point is drawn from the bridge of the process between the last known value on its left (an inspection, or the previous grid point, already drawn) and the next inspection. By the Markov property, the resampled values have the distribution of the process given the observations: under H0 and with the true parameters, the reconstructed increments are i.i.d. with the distribution of X_h, h being the step of the grid.

`"Bridge"` (Grall-Maës, 2012). Each trajectory i has its own grid $t'_k = k\, t_{i,max}/m_i$, $k = 0, \dots, m_i$, where $t_{i,max}$ is its last inspection time and m_i its number of increments: as many points as observations. The steps of two trajectories are generally different, so their reconstructed increments have different distributions: the trajectories are tested one by one.

`"ExtBridge"`. The inspection times are taken with d decimal places and the step is

$$h = 10^{-d} \gcd\big(\lfloor 10^d t \rfloor\big),$$

the gcd being taken over all the inspection times t of all the trajectories, where $\lfloor u \rfloor$ is the largest integer not greater than u. It is the largest step such that every inspection time of every trajectory is a grid point (see the help page levyBridge::levyBridge-package). All the observations are kept on the grid, and all the trajectories have the same step: one test on all of them (group `"all"`) in addition to the tests per trajectory. The grid can have many more points than observations.

`"MultiBridge"`. One grid of Grall-Maës type common to all the trajectories, of step

$$h = \frac{t_{1,max} + \dots + t_{n,max}}{m_1 + \dots + m_n},$$

the total observed duration divided by the total number of increments; the number of reconstructed increments is then about the number of observed increments. For one trajectory, $h = t_N/N$: "MultiBridge" is the method "Bridge". The grid $t'_k = kh$ does not necessarily contain the inspection times. All the trajectories have the same step: one test on all of them (group `"all"`) in addition to the tests per trajectory.

- `option = 1` (default, recommended): trajectory i is resampled on the grid points $kh \le t_{i,max}$, each one drawn from the bridge as described above. The end of the trajectory, between its last grid point and $t_{i,max}$, is not used. A trajectory shorter than h gives no reconstructed increment (with a warning).
- `option = 2`: all the trajectories are resampled up to the last inspection time over all of them, $T = \max_i t_{i,max}$, so that they all have the same number of grid points. The grid ends exactly at T: the number of steps T/h is rounded to the nearest integer N, and the step becomes

$h^* = T/N$, close to h; the grid is $t'_k = kh^*$, $k = 0, \ldots, N$. Up to $t_{i,max}$, as in the option 1; beyond, there is no observation left and no bridge: each new grid value is the previous one plus an increment of the process drawn with **all** the parameters estimated by `levyfit` (for instance the rate b of the gamma process in addition to the shape a), the first one starting from the last observation $x_{i,N}$. Under H0 with the true parameters the pooled sample is still i.i.d., but the extended values do not come from the data: they follow the fitted null model by construction and dilute the information of the observations, which lowers the power, all the more as the trajectories have different durations. Use it only for trajectories of similar durations.

Which method? One trajectory: `"Bridge"`, or `"ExtBridge"` to keep all the observations on the grid. Several trajectories with one test on all of them: `"MultiBridge"` (about as many points as observations) or `"ExtBridge"` (grid containing all the inspection times, often many more points). See also levyBridge::levyBridge-package for the grids and the bridges.

Several trajectories

Each trajectory is tested alone (group `1`, `2`, ...). When all the trajectories have the same time step, their increments have the same distribution under H0: they form one i.i.d. sample and are also tested together (group `"all"`). This is always the case with the methods `"ExtBridge"` (step common to all the trajectories, the largest one such that every inspection time of every trajectory is a grid point) and `"MultiBridge"` (one grid common to all the trajectories). With the method `"Bridge"` the step t_N/N of each trajectory is generally different, and the increments of different trajectories are then not tested together: use `"MultiBridge"` or `"ExtBridge"` for one test on all the trajectories.

Randomness

The bridges and the bootstrap are random: use `set.seed()` before `gof.levy()` to reproduce a result.

References

Grall-Maës, E. (2012). Use of the Kolmogorov-Smirnov test for gamma process. *Proceedings of the Institution of Mechanical Engineers, Part O: Journal of Risk and Reliability*, 226(6), 624-634. doi:10.1177/1748006X12462522

Stute, W., Gonzalez Manteiga, W. and Presedo Quindimil, M. (1993). Bootstrap based goodness-of-fit-tests. *Metrika*, 40, 243-256.

The references of each test are given in `tests_disponibles()`.

See Also

`test_adequation()` for an i.i.d. sample of increments, `tests_disponibles()` for the list of the tests, `levyfit::gamma_mle()` for the estimation, levyBridge::levyBridge-package for the bridges.

Examples

```
set.seed(1)

# 1. One gamma trajectory (a = 2, b = 1) inspected every time unit:
#    the observed increments are tested directly, against the true model
#    and against the other models
t <- DegradSim::inspection_times("regular", n = 100, tmax = 100)$time
x <- DegradSim::sim_gamma(t, a = 2, b = 1)
gof.levy(x, "HGP", B = 100)
gof.levy(x, "HWP", B = 100)
```

```
gof.levy(x, "InvGaus", B = 100)

# 2. 400 irregular inspections, more and more frequent (a = 2, b = 1):
#    estimation, bridge of the tested model, test. The true model is not
#    rejected, the other models are.
t2 <- DegradSim::inspection_times("decreasing", n = 400, tmax = 400, digits = 1)$time
x2 <- DegradSim::sim_gamma(t2, a = 2, b = 1)
res <- gof.levy(x2, "HGP", test = c("KS", "AD"), B = 100)
res
res$parameters
res$results
gof.levy(x2, "HWP")
gof.levy(x2, "InvGaus", test = c("KS", "AD"), B = 100)

# 2b. The same process with only 30 irregular inspections: the bridge fills
#     the gaps with the tested model, which reduces the power of the tests.
#     Over many simulations, the Wiener model is rejected much less often
#     than with the 400 inspections of example 2.
t30 <- DegradSim::inspection_times("decreasing", n = 30, tmax = 30, digits = 1)$time
x30 <- DegradSim::sim_gamma(t30, a = 2, b = 1)
gof.levy(x30, "HWP")

# 3. The same trajectory with the method "ExtBridge"
gof.levy(x2, "HGP", test = c("KS", "AD"), method = "ExtBridge", B = 100)

# 3b. A small shape (a = 2): the reconstructed increments can be tiny but stay positive
x4 <- DegradSim::sim_gamma(t2, a = 2, b = 3)
gof.levy(x4, "HGP", test = "AD", method = "ExtBridge", B = 100)

# 3c. Times that are not rounded have many decimal places: "ExtBridge" needs
#     digits, and the times are truncated to 1 decimal place (with a warning)
t6 <- DegradSim::inspection_times("decreasing", n = 20, tmax = 40)$time
x6 <- DegradSim::sim_gamma(t6, a = 10, b = 5)
try(gof.levy(x6, "HGP", test = "AD", method = "ExtBridge", B = 100))
gof.levy(x6, "HGP", test = "AD", method = "ExtBridge", B = 100, digits = 1)

# 4. Wiener model on the same data as in 2, at level 10 %
gof.levy(x2, "hwp", alpha = 0.1)

# 5. Three Wiener trajectories (mu = 0.5, sigma = 1.5) with their own
#    inspection times, up to 40, 50 and 60 (a list): one test per trajectory;
#    their negative increments are impossible for the gamma and inverse
#    Gaussian processes
plan <- DegradSim::inspection_times("dec-inc", n = 20, n_traj = 3,
                                    tmax = c(40, 50, 60), digits = 1)
liste <- DegradSim::sim_wiener(plan, mu = 0.5, sigma = 1.5)
gof.levy(liste, "HWP", test = c("SW", "AD"))
gof.levy(liste, "HGP", B = 100)
gof.levy(liste, "InvGaus", B = 100)

# 5b. The same trajectories with "ExtBridge": common step, so also one test on all
gof.levy(liste, "HWP", test = c("SW", "AD"), method = "ExtBridge")

# 5c. "MultiBridge": one common grid of Grall-Maes type, also one test on all
gof.levy(liste, "HWP", test = c("SW", "AD"), method = "MultiBridge")
gof.levy(liste, "HWP", test = c("SW", "AD"), method = "MultiBridge", option = 2)
```

```
# 6. Four trajectories with the same irregular inspection times (matrix):
#    the grids have the same step, so the increments are also tested together
m <- DegradSim::sim_gamma(t2, a = 10, b = 5, n_traj = 4)
gof.levy(m, "HGP", test = "AD", B = 100)

# 6b. Three trajectories inspected every 2 time units (regular, same step):
#     observed increments, one test per trajectory and one on all of them
m2 <- DegradSim::sim_gamma(t, a = 2, b = 3, n_traj = 3)
gof.levy(m2, "HGP", test = "VGE", B = 50)

# 7. Data from another process: H0 is rejected. In 100 simulations of each
#    case below, the tests rejected H0 in about 90 % to 100 % of the cases.
t100 <- DegradSim::inspection_times("regular", n = 100, tmax = 100)$time

# Very skewed gamma data (a = 0.5, b = 1) tested as a Wiener process
g <- DegradSim::sim_gamma(t100, a = 0.5, b = 1)
gof.levy(g, "HWP", test = c("SW", "AD", "JB"))

# Heavy-tailed variance-gamma data (nu = 2) tested as a Wiener process
v <- DegradSim::sim_vg(t100, theta = 0.5, sigma = 1.5, nu = 2)
gof.levy(v, "HWP", test = c("SW", "AD", "JB"))

# Inverse Gaussian data (mu = 1, eta = 0.5) tested as a gamma process
w <- DegradSim::sim_ig(t100, mu = 1, eta = 0.5)
gof.levy(w, "HGP", test = c("KS", "AD"), B = 100)


# 8. Inverse Gaussian (mu = 1, eta = 1) and variance-gamma
#    (theta = 0.5, sigma = 1.5, nu = 1) data on the 400 irregular times of
#    example 2, tested against several models (slower)
ig <- DegradSim::sim_ig(t2, mu = 1, eta = 1)
gof.levy(ig, "InvGaus", test = c("KS", "AD"), B = 100)
gof.levy(ig, "HGP", test = c("KS", "AD"), B = 100)
gof.levy(ig, "HWP")

vg <- DegradSim::sim_vg(t2, theta = 0.5, sigma = 1.5, nu = 1)
gof.levy(vg, "VG", test = "AD", B = 50)
gof.levy(vg, "HGP", B = 100)
gof.levy(vg, "InvGaus", B = 100)
gof.levy(vg, "HWP")

# 9. Meixner model (a = 1, b = 1, d = 0.5, m = 0.2), regular and irregular
#    inspections; with few increments meixner_mle() may give no estimate,
#    and the test then stops with a message
t_reg <- DegradSim::inspection_times("regular", n = 100, tmax = 100)$time
mx <- DegradSim::sim_meixner(t_reg, a = 1, b = 1, d = 0.5, m = 0.2)
gof.levy(mx, "Meixner", test = "AD", B = 30)
gof.levy(mx, "HWP", B = 100)

t3 <- DegradSim::inspection_times("dec-inc", n = 60, tmax = 120, digits = 1)$time
mx3 <- DegradSim::sim_meixner(t3, a = 1, b = 1, d = 0.5, m = 0.2)
try(gof.levy(mx3, "Meixner", test = "AD", B = 30))
```

---

tests_disponibles *Available goodness-of-fit tests*

Description

Lists the tests that `gof.levy()` and `test_adequation()` can apply to each model.

Usage

```
tests_disponibles(model = NULL)
```

Arguments

`model`	`NULL` (default) for all the models, or one model: `"HGP"` (gamma process), `"HWP"` (Wiener process), `"InvGaus"` (inverse Gaussian process), `"VG"` (variance-gamma process) or `"Meixner"` (Meixner process). Upper or lower case does not matter.

Value

With `model = NULL`, a named list with one vector of test names per model; otherwise the vector of test names of this model.

The tests

All the tests are tests of a composite hypothesis: the increments $\Delta x_1, \ldots, \Delta x_n$ are i.i.d. with a distribution of the family of the model, whose parameters are unknown and estimated on the sample.

Gamma process (`"HGP"`), increments $\Gamma(a\Delta, b)$:

- `"HME1"`: Henze, Meintanis and Ebner (2012), Laplace transform (statistic `gofgamma::HME1`, tuning parameter 1);
- `"BE"`: Betsch and Ebner (2019), characterization by a fixed point (statistic `gofgamma::BE`, tuning parameter 1);
- `"VGE"`: test of Villasenor and Gonzalez-Estrada (2015), ratio of two variance estimators (function `goft::gamma_test`), with a p-value by parametric bootstrap;
- `"KS"`, `"CvM"`, `"AD"`, `"Watson"`: Kolmogorov-Smirnov, Cramer-von Mises, Anderson-Darling and Watson (`gofgamma`).

For the tests HME1, BE, KS, CvM, AD and Watson, the statistic and the parametric bootstrap are those of the package `gofgamma` (the shape is estimated by `gofgamma::gamma_est`, and each bootstrap sample of the gamma distribution with this shape is estimated again); all the bootstrap statistics are kept, which gives both the p-value and the critical value.

Wiener process (`"HWP"`), increments $\mathcal{N}(\mu\Delta, \sigma^2\Delta)$: tests of normality with estimated mean and variance.

- `"AD"`, `"CvM"` and `"Lilliefors"`: Anderson-Darling, Cramer-von Mises and Lilliefors tests (package `nortest`). The Lilliefors test is the Kolmogorov-Smirnov test with estimated mean and variance: `"KS"` is also accepted for it, and the result is labelled `"Lilliefors"`;
- `"SW"`: Shapiro-Wilk (`stats::shapiro.test`);
- `"JB"`: Jarque-Bera (`tseries::jarque.bera.test`);
- `"DAgostinoPearson"`: D'Agostino-Pearson omnibus test K^2, combining skewness and kurtosis (`fBasics::dagoTest`).

Inverse Gaussian process (`"InvGaus"`), increments $IG(\mu\Delta, \eta\Delta^2)$:

- `"ABEV1"`, `"ABEV2"`: Allison, Betsch, Ebner and Visagie (2022) (`gofIG`, tuning parameter 10, maximum likelihood), characterization of the distribution function (see below);
- `"HK1"`, `"HK2"`: Henze and Klar (2002) (`gofIG`), empirical Laplace transform (see below);
- `"KS"`, `"CvM"`, `"AD"`: Kolmogorov-Smirnov, Cramer-von Mises and Anderson-Darling (`gofIG`);
- `"VGE_ratio"`: Villasenor and Gonzalez-Estrada (2015), ratio of two variance estimators, asymptotic p-value, suited to large samples (`goft::ig_test`, method `"ratio"`);
- `"VGE_gamma"`: Villasenor and Gonzalez-Estrada (2015), transformation of the data to gamma variables of shape 1/2 and Anderson-Darling test (`goft::ig_test`, method `"transf"`);
- `"BG"`: Baringhaus and Gaigall (2015) (`gofIG`), independence test (see below). It is slow and is not applied by default: give `test = "BG"`.

Test BG. For two independent positive variables X_1 and X_2 with the same distribution (with finite moments $E[X_1^2]$ and $E[1/X_1]$), the variables $Y = (X_1+X_2)/2$ and $Z = (1/X_1+1/X_2)/2-1/Y$ are independent if and only if this distribution is inverse Gaussian (Allison, Betsch, Ebner and Visagie, 2022, section 5.2.4). The test computes Y and Z for all the pairs (X_i, X_j), $i \neq j$, of the sample and measures their dependence with the Hoeffding-Blum-Kiefer-Rosenblatt criterion; H0 is rejected for large values, and the p-value is obtained by parametric bootstrap. `gofIG` compares every pair with every other pair: the cost grows as n^4, times $B + 1$. One statistic already takes a few seconds for $n = 100$: use BG only with small samples, a few dozen increments (a warning is given above 50 increments).

Tests ABEV1, ABEV2, HK1 and HK2. Let $X_1, \ldots, X_n$ be the sample, $IG(m, \lambda)$ the inverse Gaussian distribution of mean m and shape λ, $\hat{m} = \bar{X}$ and $\hat{\lambda} = 1/\overline{(1/X - 1/\bar{X})}$ the maximum likelihood estimators. The four tests use the scaled data $Y_j = X_j/\hat{m}$ and $\hat{\varphi} = \hat{\lambda}/\hat{m}$: under H0, Y_j is close to $IG(1, \varphi)$, and the distribution of the statistic only depends on φ. Each statistic is a weighted L^2 distance, H0 is rejected for large values, and the p-value is obtained by parametric bootstrap: `B` samples of $IG(1, \hat{\varphi})$ of size n, the statistic being computed again (with new estimates) on each one. `gofIG` gives only the p-value, so the column `critical.value` is `NA`.

- `"HK2"` compares the empirical Laplace transform $L_n(t) = \frac{1}{n}\sum_j e^{-tY_j}$ with the fitted one $L_{\hat{\varphi}}(t) = \exp(\hat{\varphi}(1 - \sqrt{1 + 2t/\hat{\varphi}}))$:

$$HK2 = n \int_0^\infty (L_n(t) - L_{\hat{\varphi}}(t))^2 \, dt.$$

- `"HK1"` uses the differential equation $L(t) + \sqrt{1 + 2t/\varphi}\, L'(t) = 0$, which characterizes the Laplace transform of $IG(1, \varphi)$; the statistic measures how far L_n is from satisfying it:

$$HK1 \propto n \int_0^\infty \left(L_n(t) + \sqrt{1 + 2t/\hat{\varphi}}\, L_n'(t)\right)^2 e^{-at} \, dt,$$

  with $a = 0$.

- `"ABEV1"` and `"ABEV2"` use a characterization of the distribution function: with the score $\tau(y) = -f'(y)/f(y) = \frac{1}{2}(\varphi + 3/y - \varphi/y^2)$ of the density f of $IG(1, \varphi)$, a positive variable Y has this distribution if and only if $F(t) = E[\tau(Y)\min(Y, t)]$ for all $t > 0$. With F_n the empirical distribution function,

$$ABEV = n \int_0^\infty \left(\frac{1}{n}\sum_j \hat{\tau}(Y_j)\min(Y_j, t) - F_n(t)\right)^2 w(t) \, dt,$$

  with the weight $w(t) = e^{-at}$ for ABEV1 and $w(t) = e^{-at^2}$ for ABEV2, and $a = 10$.

The four statistics are double sums over the sample, computed with R loops: their cost grows as n^2, times $B + 1$ for the bootstrap. For $n = 300$ and `B = 200`, one test takes about 8 seconds for HK2, 2 minutes for ABEV1 and HK1, and 7 minutes for ABEV2. With many increments (for instance the method `"ExtBridge"` of `gof.levy()`), prefer HK2 or the other tests.

Variance-gamma (`"VG"`) and Meixner (`"Meixner"`) processes: `"KS"`, `"CvM"`, `"AD"` and `"Watson"`, the Kolmogorov-Smirnov, Cramer-von Mises, Anderson-Darling and Watson statistics (Stephens, 1986). The parameters are estimated by maximum likelihood with the package `levyfit` (functions `levyfit::vg_mle()` and `levyfit::meixner_mle()`); the distribution function is computed by numerical integration of the density, and the p-value is obtained by parametric bootstrap (Stute, Gonzalez Manteiga and Presedo Quindimil, 1993). With a small sample, `levyfit::meixner_mle()` may give no estimate (see the section Algorithm of its help): the test is then not computed, with this message.

References

Gamma distribution

Henze, N., Meintanis, S. G. and Ebner, B. (2012). Goodness-of-fit tests for the Gamma distribution based on the empirical Laplace transform. *Communications in Statistics - Theory and Methods*, 41(9), 1543-1556. doi:10.1080/03610926.2010.542851 (tests HME1, KS, CvM, AD and Watson of `gofgamma`)

Betsch, S. and Ebner, B. (2019). A new characterization of the Gamma distribution and associated goodness of fit tests. *Metrika*, 82(7), 779-806. doi:10.1007/s00184019007087 (test BE)

Villasenor, J. A. and Gonzalez-Estrada, E. (2015). A variance ratio test of fit for Gamma distributions. *Statistics and Probability Letters*, 96, 281-286. doi:10.1016/j.spl.2014.10.001 (test VGE)

Normal distribution (Wiener process)

Shapiro, S. S. and Wilk, M. B. (1965). An analysis of variance test for normality (complete samples). *Biometrika*, 52(3-4), 591-611. doi:10.1093/biomet/52.34.591 (test SW)

Royston, P. (1995). Remark AS R94: A remark on Algorithm AS 181: The W test for normality. *Applied Statistics*, 44, 547-551. doi:10.2307/2986146 (computation of the test SW in `stats::shapiro.test`)

Lilliefors, H. W. (1967). On the Kolmogorov-Smirnov test for normality with mean and variance unknown. *Journal of the American Statistical Association*, 62(318), 399-402. doi:10.1080/01621459.1967.10482916 (test Lilliefors)

Dallal, G. E. and Wilkinson, L. (1986). An analytic approximation to the distribution of Lilliefors' test for normality. *The American Statistician*, 40, 294-296. (p-value of the test Lilliefors)

Stephens, M. A. (1986). Tests based on EDF statistics. In D'Agostino, R. B. and Stephens, M. A. (eds.), *Goodness-of-Fit Techniques*, Marcel Dekker, New York. (tests AD and CvM with estimated parameters, and the EDF statistics of the VG and Meixner tests)

Jarque, C. M. and Bera, A. K. (1987). A test for normality of observations and regression residuals. *International Statistical Review*, 55(2), 163-172. doi:10.2307/1403192 (test JB)

D'Agostino, R. B. and Pearson, E. S. (1973). Tests for departure from normality. Empirical results for the distributions of b_2 and $\sqrt{b_1}$. *Biometrika*, 60(3), 613-622. doi:10.1093/biomet/60.3.613 (test DAgostinoPearson)

Inverse Gaussian distribution

Allison, J. S., Betsch, S., Ebner, B. and Visagie, I. J. H. (2022). On testing the adequacy of the inverse Gaussian distribution. *Mathematics*, 10(3), 350. doi:10.3390/math10030350 (tests ABEV1, ABEV2, KS, CvM and AD of `gofIG`)

Henze, N. and Klar, B. (2002). Goodness-of-fit tests for the inverse Gaussian distribution based on the empirical Laplace transform. *Annals of the Institute of Statistical Mathematics*, 54(2), 425-444. doi:10.1023/A:1022442506681 (tests HK1 and HK2)

Villasenor, J. A. and Gonzalez-Estrada, E. (2015). Tests of fit for inverse Gaussian distributions. *Statistics and Probability Letters*, 105, 189-194. doi:10.1016/j.spl.2015.06.017 (tests VGE_ratio and VGE_gamma)

Baringhaus, L. and Gaigall, D. (2015). On an independence test approach to the goodness-of-fit problem. *Journal of Multivariate Analysis*, 140, 193-208. (test BG)

EDF statistics and parametric bootstrap (VG and Meixner)

Anderson, T. W. and Darling, D. A. (1954). A test of goodness of fit. *Journal of the American Statistical Association*, 49, 765-769.

Watson, G. S. (1961). Goodness-of-fit tests on a circle. *Biometrika*, 48(1-2), 109-114.

Stute, W., Gonzalez Manteiga, W. and Presedo Quindimil, M. (1993). Bootstrap based goodness-of-fit-tests. *Metrika*, 40, 243-256.

See Also

`gof.levy()`, `test_adequation()`

Examples

```
# All the models and their tests
tests_disponibles()

# The tests of one model (upper or lower case)
tests_disponibles("HGP")
tests_disponibles("meixner")
```

---

test_adequation *Goodness-of-fit tests for an i.i.d. sample of increments*

Description

Applies goodness-of-fit tests to a sample of increments of a Levy process observed with a constant time step: the increments are then independent and identically distributed. This is the last step of `gof.levy()`, also useful alone when the increments are already available (regular inspections, or increments reconstructed by the package `levyBridge`).

Usage

```
test_adequation(dx, model, test = NULL, alpha = 0.05, B = 200)
```

Arguments

`dx`	Numeric vector of at least 3 increments, all over the same time step.
`model`	`"HGP"` (gamma process), `"HWP"` (Wiener process), `"InvGaus"` (inverse Gaussian process), `"VG"` (variance-gamma process) or `"Meixner"` (Meixner process). Upper or lower case does not matter.
`test`	Names of the tests to apply; `NULL` (default) applies all the tests of the model except the slow test BG of the inverse Gaussian model, see `tests_disponibles()`.

alpha	Level of the tests, strictly between 0 and 1.
B	Number of bootstrap replications, for the tests whose p-value or critical value is computed by parametric bootstrap.

Details

The null hypothesis is: the increments are i.i.d. with a distribution of the family of the model (parameters unknown). Under the gamma and inverse Gaussian models the increments are positive with probability one: a negative or zero increment is impossible under H0, and all the tests then reject H0 with a p-value 0, without computation.

A test that fails (for instance a sample too small for the estimation) does not stop the others: its row has the decision `"not computed"` and the error message.

The tests are described in `tests_disponibles()`. The bootstrap tests are random: use `set.seed()` to reproduce a result.

Value

A data frame with one row per test and the columns:

- `test`: name of the test;
- `statistic`: value of the test statistic;
- `p.value`: p-value; the test rejects H0 when `p.value <= alpha`;
- `critical.value`: for the tests whose p-value is computed by parametric bootstrap, the quantile of order `1 - alpha` of the bootstrap statistics (H0 is rejected when the statistic exceeds it); `NA` when it is not available (tests with an exact or asymptotic p-value, and tests of `gofIG`, which give only the p-value);
- `decision`: `"reject H0"`, `"do not reject H0"`, or `"not computed"` when the test failed;
- `message`: reason of a failure or of a certain rejection, `""` otherwise.

The column `message` is not printed when it is empty for all the tests; it stays in the data frame. When H0 is rejected for sure (negative or zero increment under the gamma or inverse Gaussian model), `print()` shows this conclusion in one line; the data frame keeps one row per test.

References

Gamma distribution

Henze, N., Meintanis, S. G. and Ebner, B. (2012). Goodness-of-fit tests for the Gamma distribution based on the empirical Laplace transform. *Communications in Statistics - Theory and Methods*, 41(9), 1543-1556. doi:10.1080/03610926.2010.542851 (tests HME1, KS, CvM, AD and Watson of `gofgamma`)

Betsch, S. and Ebner, B. (2019). A new characterization of the Gamma distribution and associated goodness of fit tests. *Metrika*, 82(7), 779-806. doi:10.1007/s00184019007087 (test BE)

Villasenor, J. A. and Gonzalez-Estrada, E. (2015). A variance ratio test of fit for Gamma distributions. *Statistics and Probability Letters*, 96, 281-286. doi:10.1016/j.spl.2014.10.001 (test VGE)

Normal distribution (Wiener process)

Shapiro, S. S. and Wilk, M. B. (1965). An analysis of variance test for normality (complete samples). *Biometrika*, 52(3-4), 591-611. doi:10.1093/biomet/52.34.591 (test SW)

Royston, P. (1995). Remark AS R94: A remark on Algorithm AS 181: The W test for normality. *Applied Statistics*, 44, 547-551. doi:10.2307/2986146 (computation of the test SW in `stats::shapiro.test`)

Lilliefors, H. W. (1967). On the Kolmogorov-Smirnov test for normality with mean and variance unknown. *Journal of the American Statistical Association*, 62(318), 399-402. doi:10.1080/01621459.1967.10482916 (test Lilliefors)

Dallal, G. E. and Wilkinson, L. (1986). An analytic approximation to the distribution of Lilliefors' test for normality. *The American Statistician*, 40, 294-296. (p-value of the test Lilliefors)

Stephens, M. A. (1986). Tests based on EDF statistics. In D'Agostino, R. B. and Stephens, M. A. (eds.), *Goodness-of-Fit Techniques*, Marcel Dekker, New York. (tests AD and CvM with estimated parameters, and the EDF statistics of the VG and Meixner tests)

Jarque, C. M. and Bera, A. K. (1987). A test for normality of observations and regression residuals. *International Statistical Review*, 55(2), 163-172. doi:10.2307/1403192 (test JB)

D'Agostino, R. B. and Pearson, E. S. (1973). Tests for departure from normality. Empirical results for the distributions of b_2 and $\sqrt{b_1}$. *Biometrika*, 60(3), 613-622. doi:10.1093/biomet/60.3.613 (test DAgostinoPearson)

Inverse Gaussian distribution

Allison, J. S., Betsch, S., Ebner, B. and Visagie, I. J. H. (2022). On testing the adequacy of the inverse Gaussian distribution. *Mathematics*, 10(3), 350. doi:10.3390/math10030350 (tests ABEV1, ABEV2, KS, CvM and AD of `gofIG`)

Henze, N. and Klar, B. (2002). Goodness-of-fit tests for the inverse Gaussian distribution based on the empirical Laplace transform. *Annals of the Institute of Statistical Mathematics*, 54(2), 425-444. doi:10.1023/A:1022442506681 (tests HK1 and HK2)

Villasenor, J. A. and Gonzalez-Estrada, E. (2015). Tests of fit for inverse Gaussian distributions. *Statistics and Probability Letters*, 105, 189-194. doi:10.1016/j.spl.2015.06.017 (tests VGE_ratio and VGE_gamma)

Baringhaus, L. and Gaigall, D. (2015). On an independence test approach to the goodness-of-fit problem. *Journal of Multivariate Analysis*, 140, 193-208. (test BG)

EDF statistics and parametric bootstrap (VG and Meixner)

Anderson, T. W. and Darling, D. A. (1954). A test of goodness of fit. *Journal of the American Statistical Association*, 49, 765-769.

Watson, G. S. (1961). Goodness-of-fit tests on a circle. *Biometrika*, 48(1-2), 109-114.

Stute, W., Gonzalez Manteiga, W. and Presedo Quindimil, M. (1993). Bootstrap based goodness-of-fit-tests. *Metrika*, 40, 243-256.

See Also

`gof.levy()` for a trajectory or several trajectories, `tests_disponibles()` for the list of the tests.

Examples

```
set.seed(1)

# 1. Gamma increments (time step 2, a = 2, b = 3): all the tests of the model
dx <- rgamma(40, shape = 2 * 2, rate = 3)
test_adequation(dx, "HGP", B = 100)

# 2. Some tests only, at level 10 %
test_adequation(dx, "HGP", test = c("KS", "AD"), alpha = 0.1, B = 100)

# 3. Normal increments: tests of the Wiener model
test_adequation(rnorm(50, mean = 1, sd = 2), "HWP")
```

```
# "KS" is accepted for the Wiener model: it is the Lilliefors test
test_adequation(rnorm(50, mean = 1, sd = 2), "HWP", test = c("KS", "AD"))

# 4. The same normal increments are negative: the gamma model is rejected for sure
test_adequation(rnorm(50), "HGP", test = c("KS", "AD"))

# 5. Inverse Gaussian increments
dx_ig <- 1 / rgamma(40, 3) # not inverse Gaussian
test_adequation(dx_ig, "invgaus", test = c("KS", "AD", "VGE_ratio"), B = 100)

# The test BG is slow and is not applied by default: a small sample
test_adequation(dx_ig[1:30], "invgaus", test = "BG", B = 50)


# 6. Variance-gamma model (slower: estimation and numerical integration
#    in each bootstrap replication)
g <- rgamma(100, shape = 1, scale = 1)
dx_vg <- 0.5 * g + 1.5 * sqrt(g) * rnorm(100)
test_adequation(dx_vg, "VG", test = "AD", B = 30)

# 7. Meixner model (a = 1, b = 1, d = 0.5, m = 0.2)
set.seed(1)
gen <- Runuran::pinvd.new(Runuran::udmeixner(alpha = 1, beta = 1, delta = 0.5, mu = 0.2))
test_adequation(Runuran::ur(gen, 100), "Meixner", test = c("KS", "AD"), B = 30)
```

Index

∗ **package**
levygof-package, 2

gof.levy, 4
gof.levy(), *2, 3*, *10*, *12, 13*, *15*

levyBridge::levyBridge-package, *4–7*
levyfit::gamma_mle(), *7*
levyfit::meixner_mle(), *12*
levyfit::vg_mle(), *12*
levygof *(*levygof-package*)*, 2
levygof-package, 2

set.seed(), *7*, *14*

test_adequation, 13
test_adequation(), *2, 3*, *5–7*, *10*, *13*
tests_disponibles, 9
tests_disponibles(), *2–4*, *6, 7*, *13–15*